

For personal use only,
please do not
distribute on
non-UBC domains.

03a - Refactoring

CPSC310

Nick Bradley & Kyle Chin



Design Analysis Questions

- Meant to assess understanding of implementation work
- Responses graded on relevance and specificity
 - Vague or unclear answers will score lower than clear answers
 - Long answers aren't necessarily better than short answers

"I edited a few files and it was confusing"

-VS-

"I added the field to the constructor and the two JSON methods, ran it, and it worked. Then I restarted the server and the value was gone, which sent me to a fourth place I hadn't found yet"



Learning goals

By the end of the lecture you should be able to:

- Identify structural code issues that block new features.
- Design alternative code structures that more easily accommodate change.
- Execute safe, behavior-preserving transformations using disciplined refactoring.

Emergent Design

Any time you are trying to make a change, and it is difficult because either...

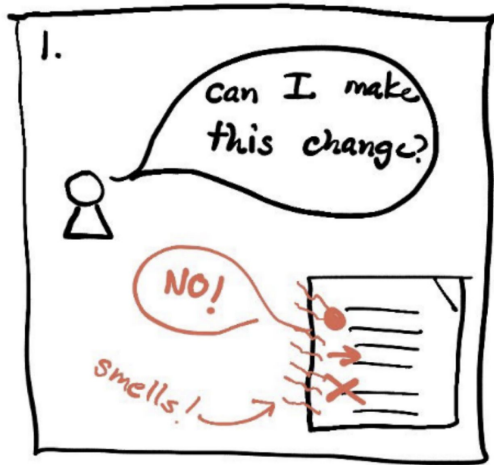
- You have to make that change in **more than one place**.
- Changes (you can tell) will result in **merge conflicts** or changes to the same file as your teammate.
- The code is too **obfuscated** to be clearly understood.

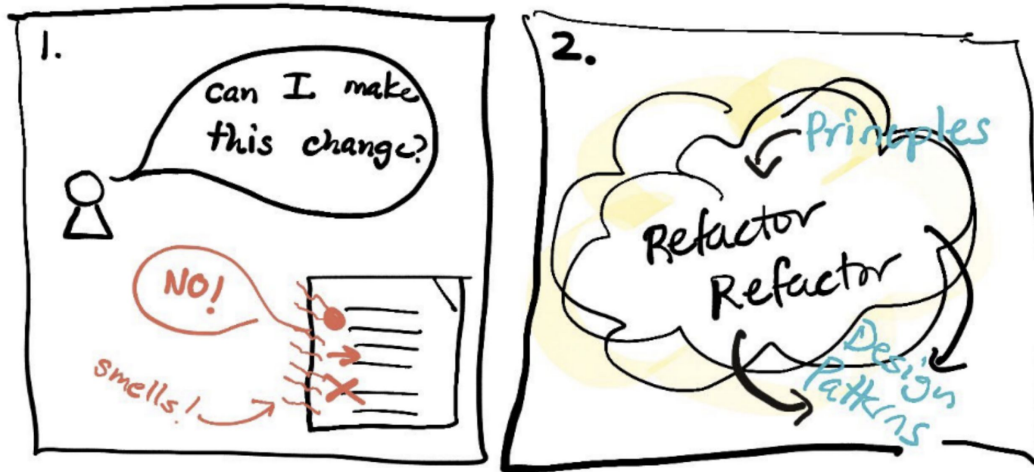
...you run the risk of introducing bugs!

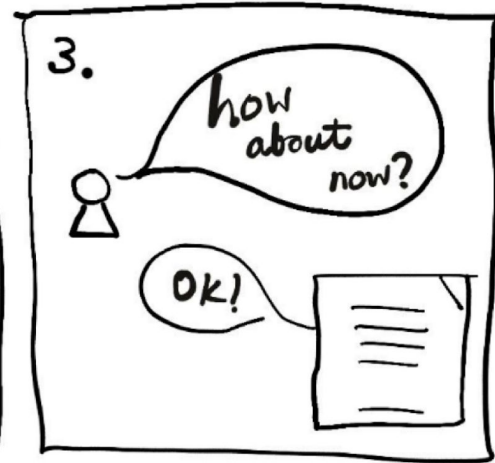
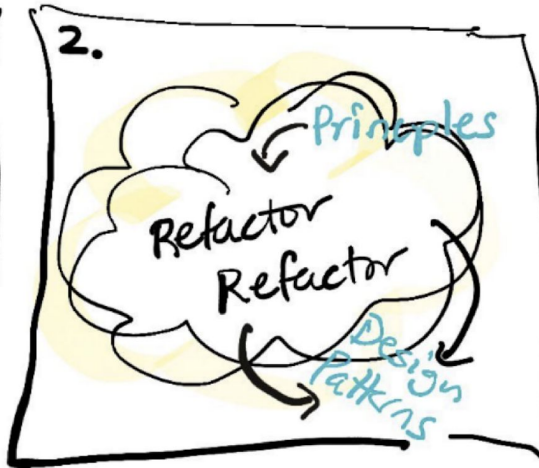
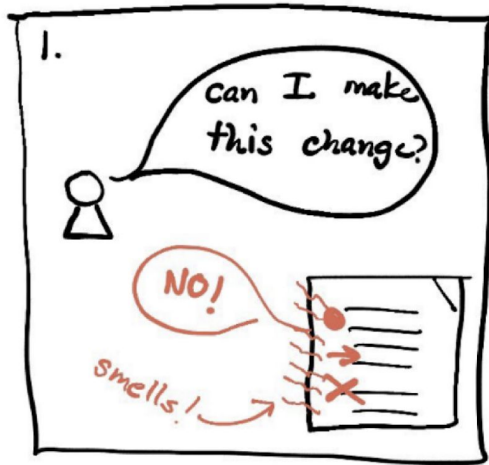
Emergent Design

To avoid this: Refactor **before** you make the change by **introducing an abstraction** to either:

- **Solve** duplication/clone/scattering that causes the multiple change locations.
- **Reduce** tangling between the pieces of functionality by splitting them up.
- **Increase** code readability to make it more understandable.







What is refactoring?

Predictable and
meaning* preserving
code transformations.

*meaning == semantics == behaviour

Behaviour in Wrong Place (Feature Envy)

When you find yourself wishing you had access to lots of fields from another class, the method you're implementing should probably live in that class.

```
class Cat {  
    private fido = new Dog();  
    public chaseDog() {  
        this.fido.run();  
        this.fido.tail.wag();  
        this.fido.mouth.smile();  
        // ... etc  
    }  
}
```

Law of Demeter: Too many dereferences mean the method is probably in the wrong spot.

Why is **Cat** telling **Dog** how to behave? Objects should define their own behaviour.

Behaviour in Wrong Place (Feature Envy)

When you find yourself wishing you had access to lots of fields from another class, the method you're implementing should probably live in that class.

```
class Cat {  
    private fido = new Dog();  
    public chaseDog() {  
        this.fido.run();  
        this.fido.tail.wag();  
        this.fido.mouth.smile();  
        // ... etc  
    }  
}
```

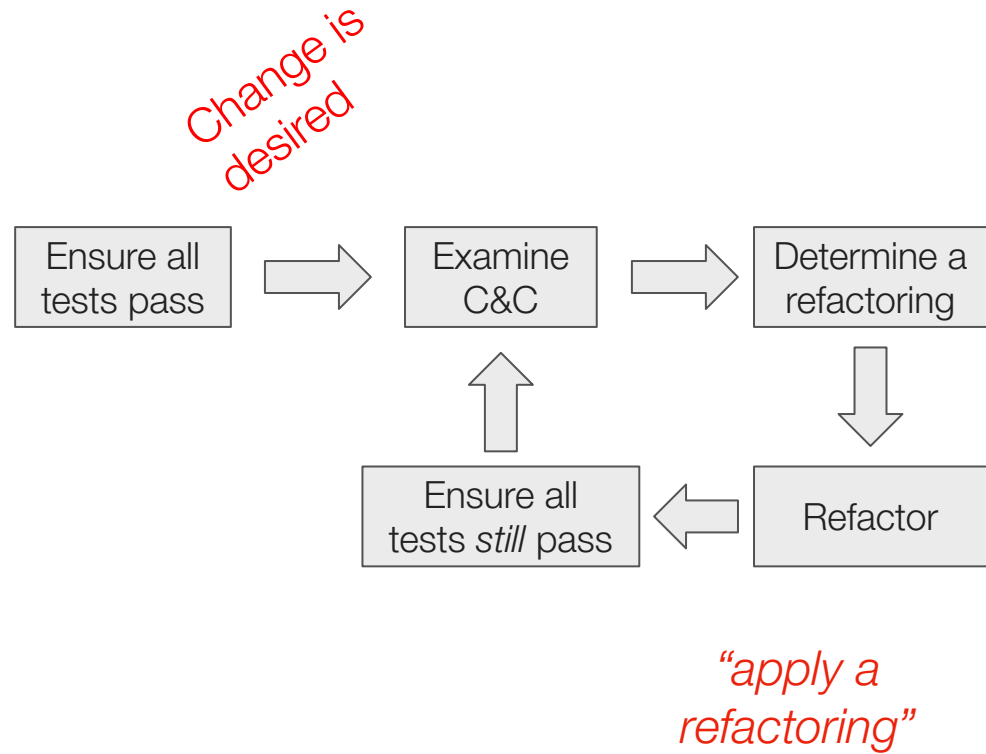
Law of Demeter: Too many dereferences mean the method is probably in the wrong spot.

```
class Cat {  
    private fido = new Dog();  
    public chaseDog() {  
        this.fido.beChased();  
    }  
}
```



How to Refactor?

1. Make sure all your tests pass.
2. Examine coupling & cohesion.
3. Determine how to refactor code.
4. Apply the refactoring.
5. Run tests to make sure you didn't break anything.
6. Repeat until change is localized.



Magic Values

Semantically-important values are encoded in the program text.

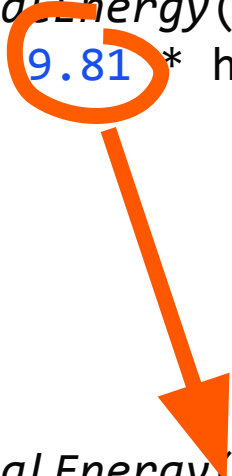
Challenges:

- Coupling (Connascence of Value)
 - Where are all the places this value has to agree?
 - What are the implications of changing it?
- Readability
 - What does this value mean?
 - Same value encoded different ways

How do we address magic values?

```
function potentialEnergy(mass: number, height: number): number {  
    return mass * 9.81 * height;  
}
```

“Replace magic number with symbol” is a typical refactor, but not the only one!



```
function potentialEnergy(mass: number, height: number): number {  
    const GRAVITATIONAL_CONSTANT = 9.81;  
    return mass * GRAVITATIONAL_CONSTANT * height;  
}
```

(Almost) Duplicate Code

The same semantic intent or computational logic is expressed across multiple locations in the program.

Challenges:

- Coupling (esp. Connascence of Algorithm)
 - Multiple sites must remain in agreement
 - If not identical, can be *very* hard to locate all duplicates
 - Hard to localize source of behaviour

Identify duplicate behaviour

```
abstract class CollectionsReport {  
    public abstract printReport(): void;  
}
```

```
class VerboseReport extends CollectionsReport {  
    public printReport(): void {  
        console.log("===Start===");  
        console.log("We are so sorry to tell you...");  
        console.log("Here's the thing");  
        console.log("We might need that money back");  
        console.log("I know, we're sorry");  
        console.log("===End===");  
    }  
}
```

duplicate

unique

```
class ConciseReport extends CollectionsReport {  
    public printReport(): void {  
        console.log("===Start===");  
        console.log("Give us our money");  
        console.log("===End===");  
    }  
}
```

Pull up duplicate behaviour

```
abstract class CollectionsReport {  
    public abstract printReport(): void;  
}
```

```
class VerboseReport extends CollectionsReport {  
    public printReport(): void {  
        console.log("===Start===");  
        console.log("We are so sorry to tell you...");  
        console.log("Here's the thing");  
        console.log("We might need that money back");  
        console.log("I know, we're sorry");  
        console.log("===End===");  
    }  
}
```

```
class ConciseReport extends CollectionsReport {  
    public printReport(): void {  
        console.log("===Start===");  
        console.log("Give us our money");  
        console.log("===End===");  
    }  
}
```

Introduce *Template Method* that calls unique functionality.

```
class VerboseReport extends CollectionsReport {  
    public printContents(): void {  
        console.log("We are so sorry to tell you...");  
        console.log("Here's the thing");  
        console.log("We might need that money back");  
        console.log("I know, we're sorry");  
    }  
}
```

```
abstract class CollectionsReport {  
    public printReport(): void {  
        console.log("===Start===");  
        this.printContents();  
        console.log("===End===");  
    }  
    public abstract printContents(): void;  
}
```

```
class ConciseReport extends CollectionsReport {  
    public printContents(): void {  
        console.log("Give us our money");  
    }  
}
```

Don't keep the pattern in the subtypes

The problem here, is that there is nothing FORCING the pattern to remain the same

```
class VerboseReport extends CollectionsReport {  
    public printReport(): void {  
        super.printHeader();  
        console.log("We are so sorry to tell you...");  
        console.log("Here's the thing");  
        console.log("We might need that money back");  
        console.log("I know, we're sorry");  
        super.printFooter();  
    }  
}
```

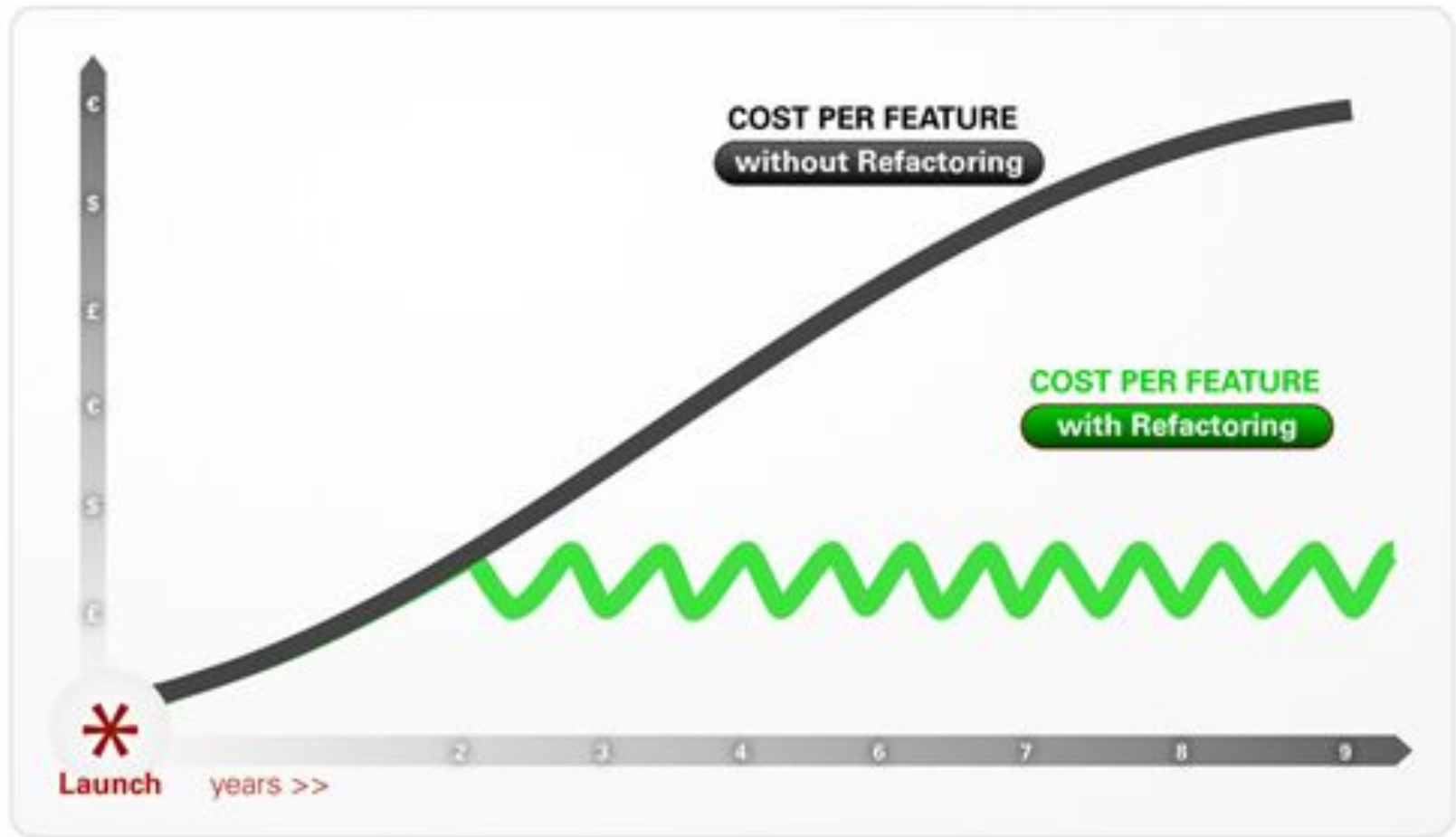
```
abstract class CollectionsReport {  
    public printHeader(): void {  
        console.log("===Start===");  
    }  
    public printFooter(): void {  
        console.log("===End===");  
    }  
    public abstract printReport(): void;  
}
```

```
class ConciseReport extends CollectionsReport {  
    public printReport(): void {  
        super.printHeader();  
        console.log("Give us our money");  
        super.printFooter();  
    }  
}
```

Refactoring addresses Technical Debt

- *Technical debt* = extra cost to change the system later because of earlier shortcuts.
- It accumulates invisibly, like interest.
- Not all technical debt is bad.
- Bad TD is the kind that slows teams down or increases risk.
- Smells → hint where debt is located.

Rule of thumb: “If you feel pain making a change, you’re probably paying interest on technical debt.”



Refactoring timeline

- NOT:
 - 2 weeks of every 6 months.
 - When the tests are failing.
 - When you should just rewrite the code.
 - When you fix a bug.
- Instead, opportunistically during development:
 - If you recognize a warning sign.
 - Before/after adding a feature.
 - When you review code/your code is reviewed.

Learning goals

By the end of the lecture you should be able to:

- Identify structural code issues that block new features.
- Design alternative code structures that more easily accommodate change.
- Execute safe, behavior-preserving transformations using disciplined refactoring.

Refactoring Recap

- Refactoring changes code representation without altering external behavior.
- Different code smells point to different structural fixes (e.g., Feature Envy, Magic Values, Duplicate Code).
- Refactoring controls the long-term cost of change by continuously reducing technical debt.
- Always follow a procedure: verify passing tests before you touch the code, and after every single change.